

[Level: Overview]

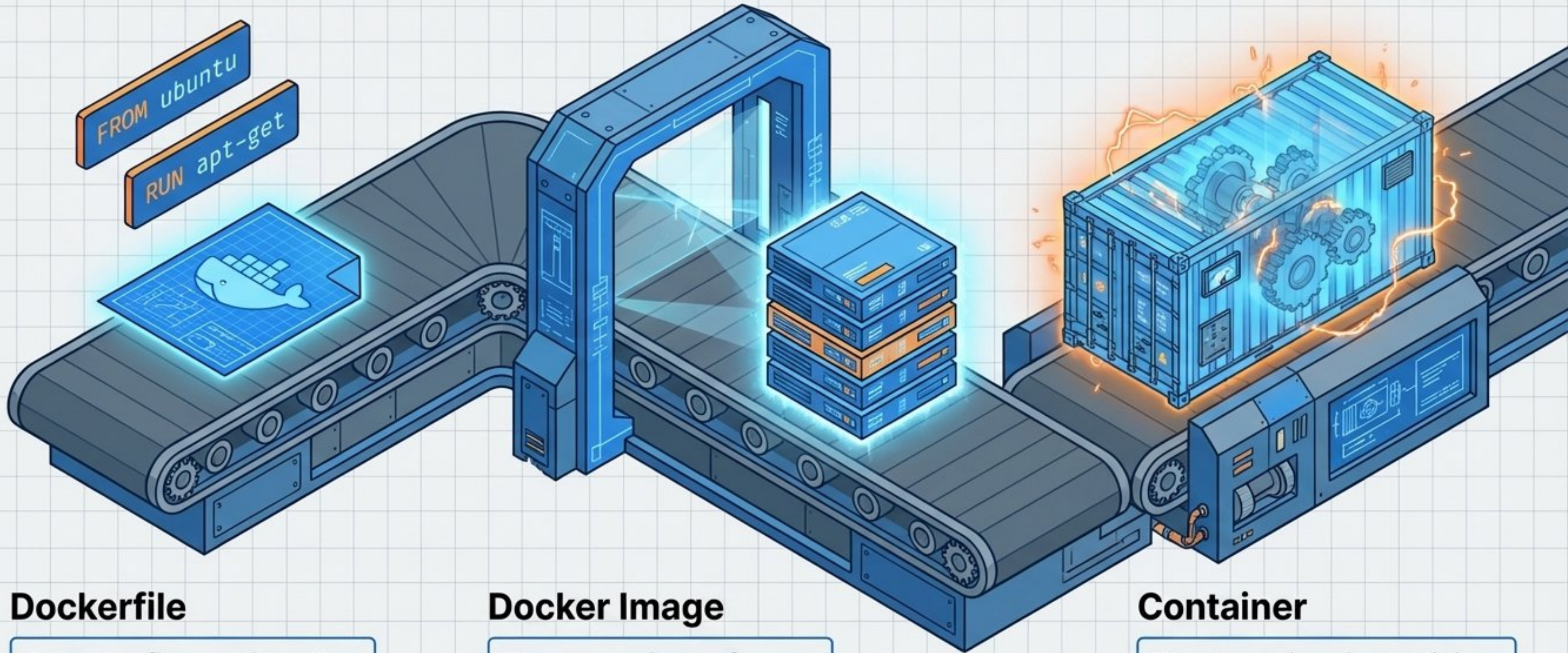


Engineering the Modern Digital Shipyard

A Masterclass in Dockerfiles, Images, and Containers.

The Assembly Line: Build, Ship, Run

[Level: Pipeline]



Dockerfile

A Dockerfile provides the raw instructions.

Docker Image

The Image is the frozen, packaged snapshot.

Container

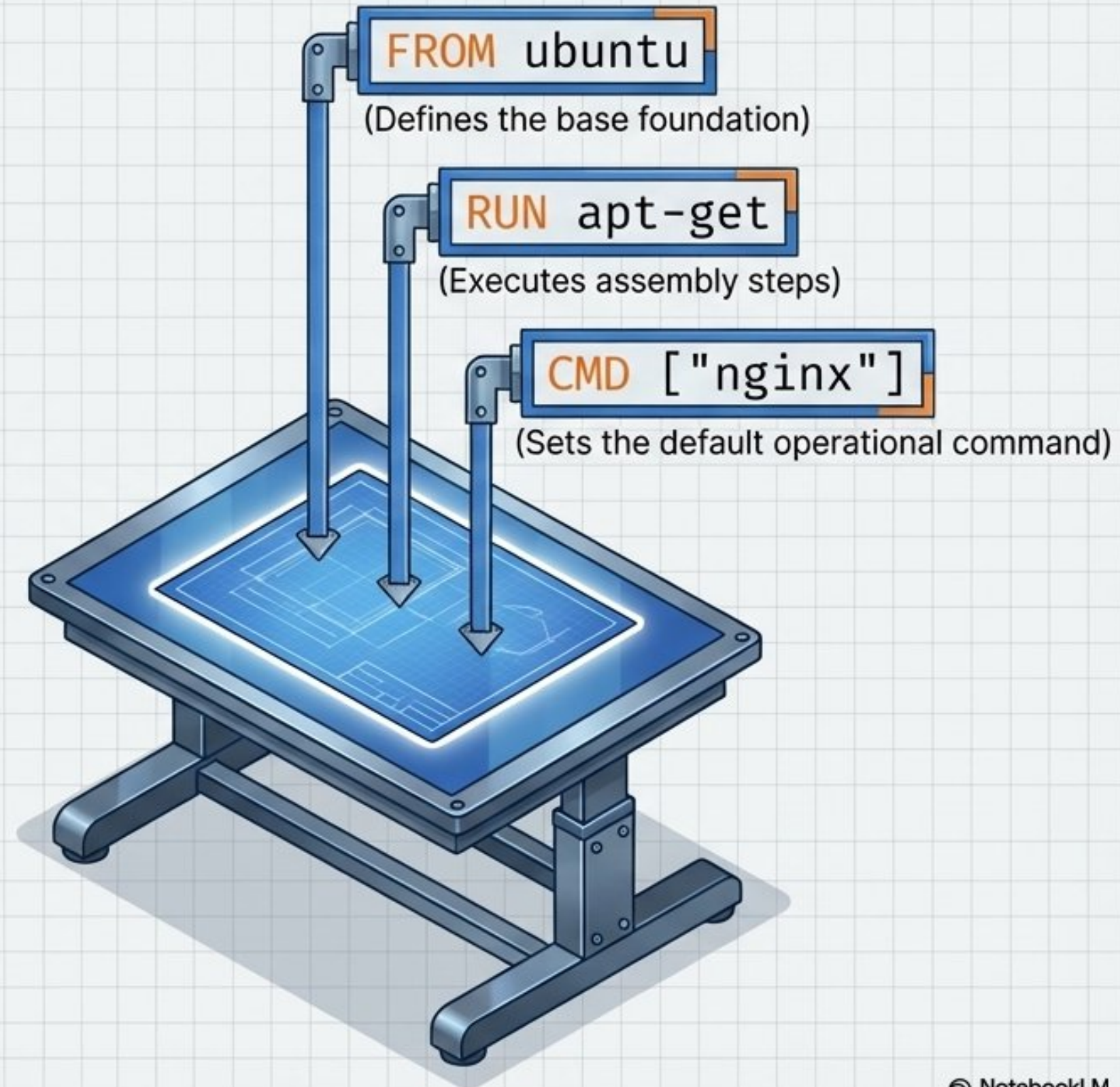
The Container is the living, running application.

Phase 1: The Blueprint (Dockerfile)

[Level: Stage 1 - Build]

A Dockerfile is a simple text file containing raw, declarative instructions needed to assemble an environment.

It is the ultimate manifestation of Infrastructure as Code.

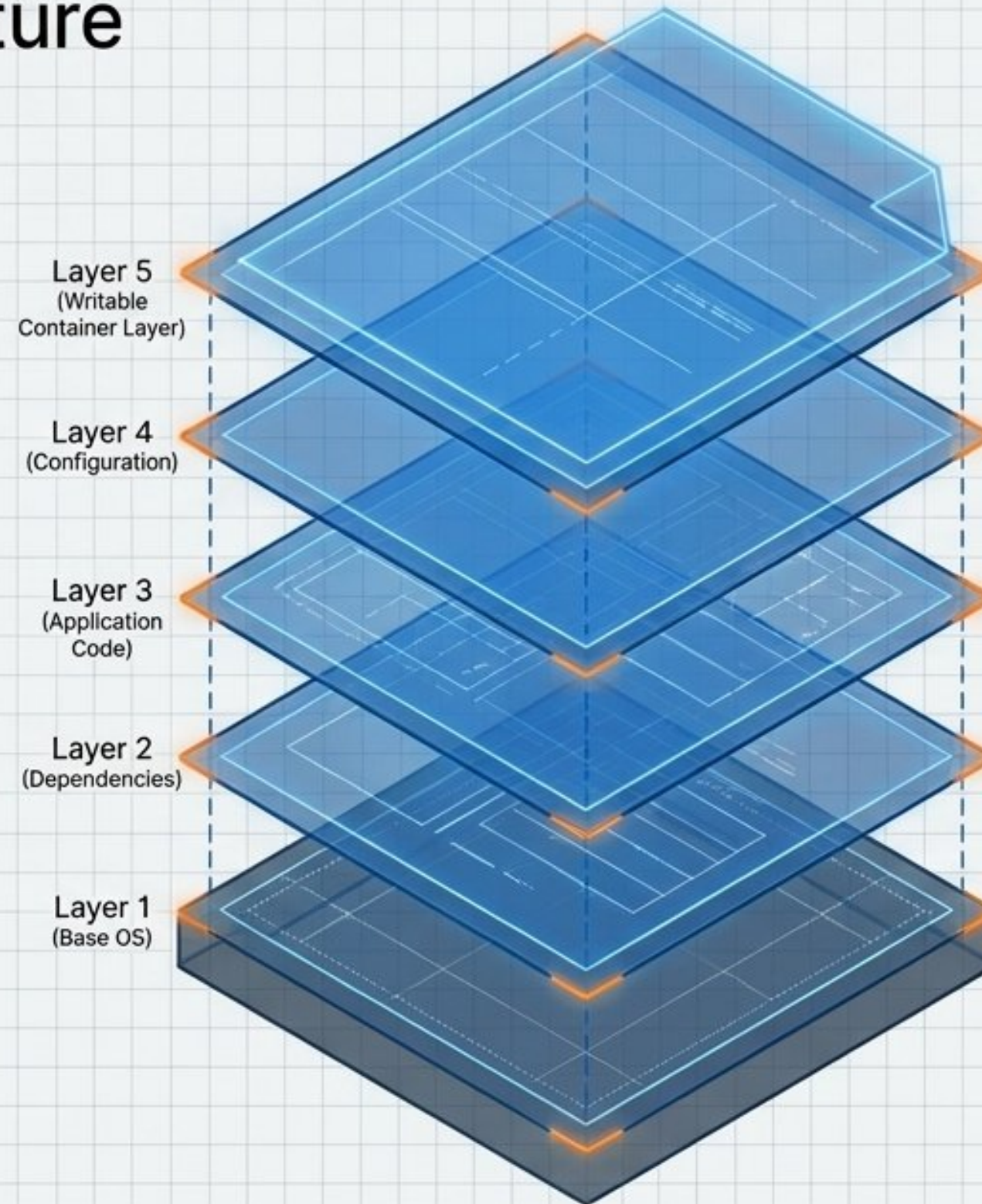


The Anatomy of a Blueprint: Layered Architecture

[Level: Stage 1 - Build]

The Mechanism

Images are built in layers. Every single instruction in the Dockerfile corresponds to a specific, discrete layer in the final build.



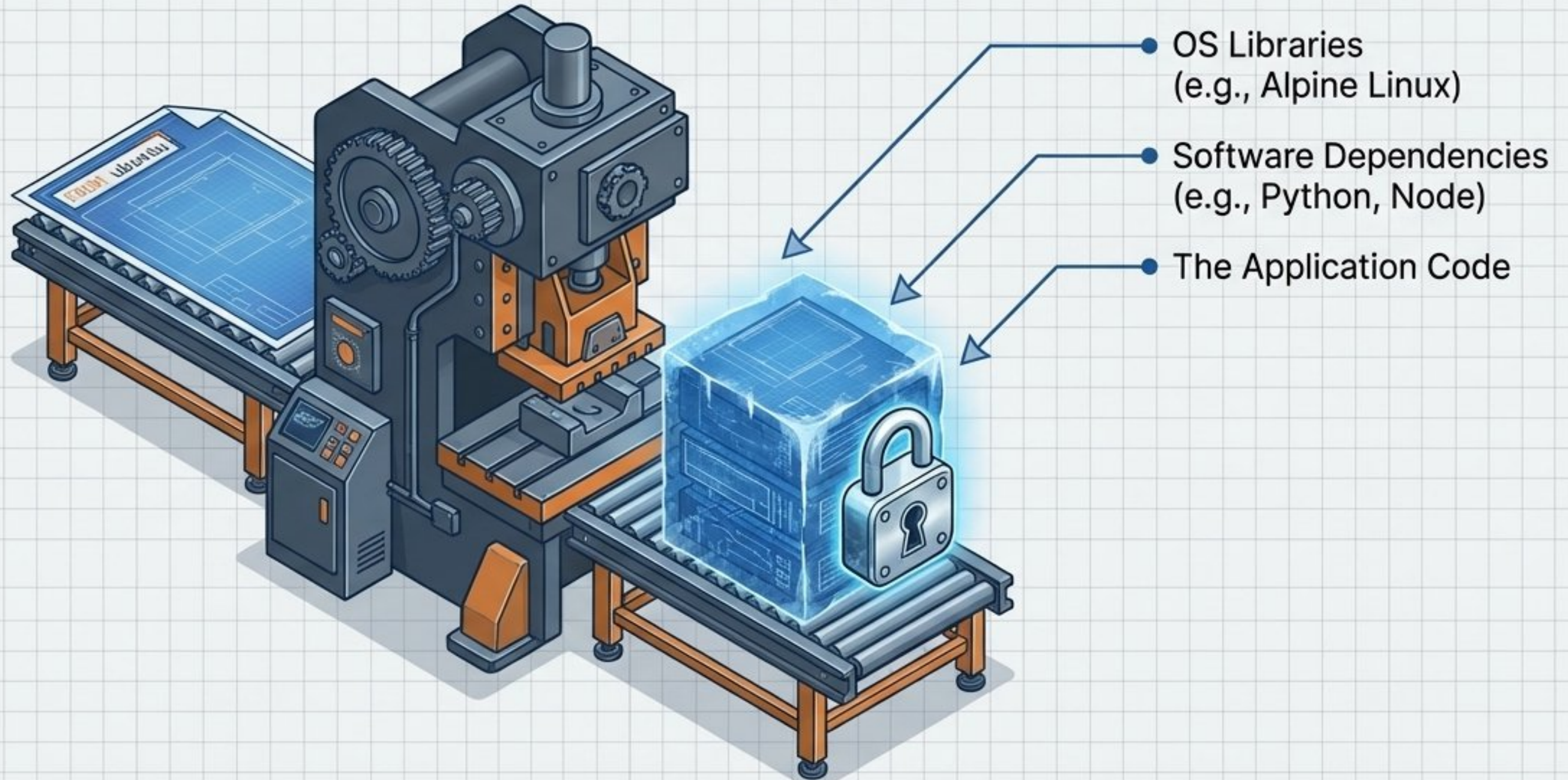
The Advantage

This layered architecture makes builds and distribution incredibly efficient. If a single line of code changes, Docker only rebuilds that specific layer, utilizing a cached version for the rest.

Phase 2: The Snapshot (Docker Image)

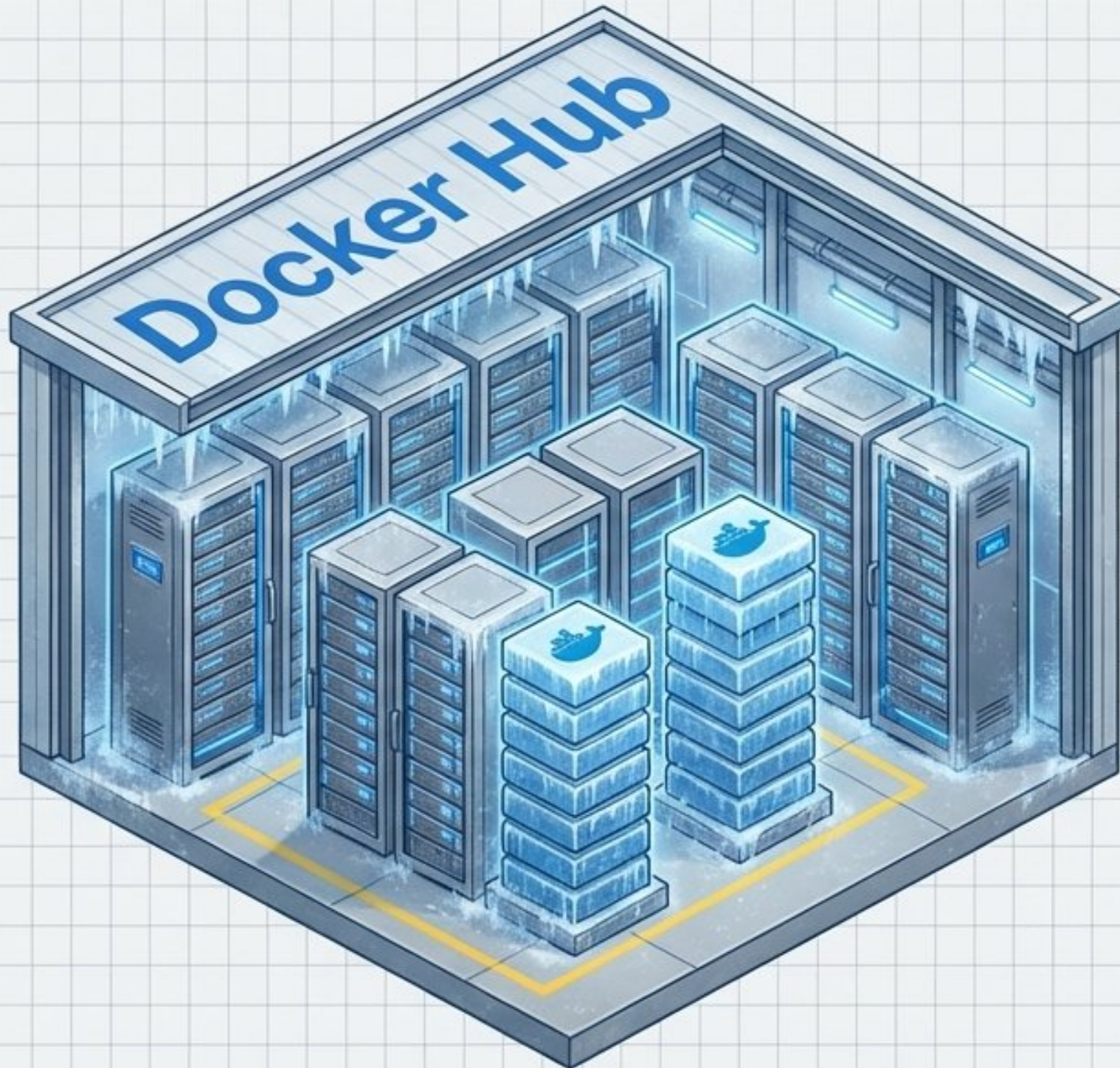
[Level: Stage 2 - Ship]

An Image is a read-only, inert template. It is the packaged, binary output of the Dockerfile.



The Global Digital Library

[Level: Registry]



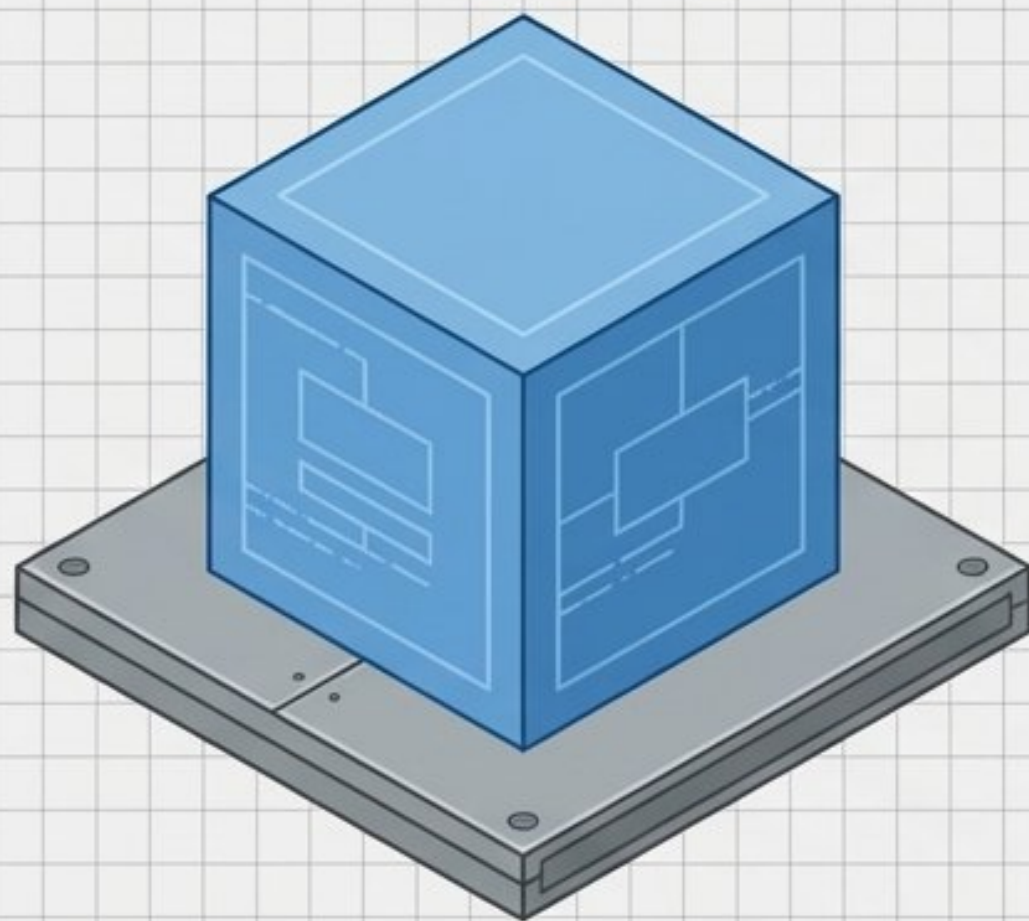
Docker Hub is the official cloud-based registry for storing, sharing, and downloading Docker images.

Instead of configuring complex software from scratch, engineers pull pre-built, standardized environments directly into their infrastructure using the command `docker pull`.

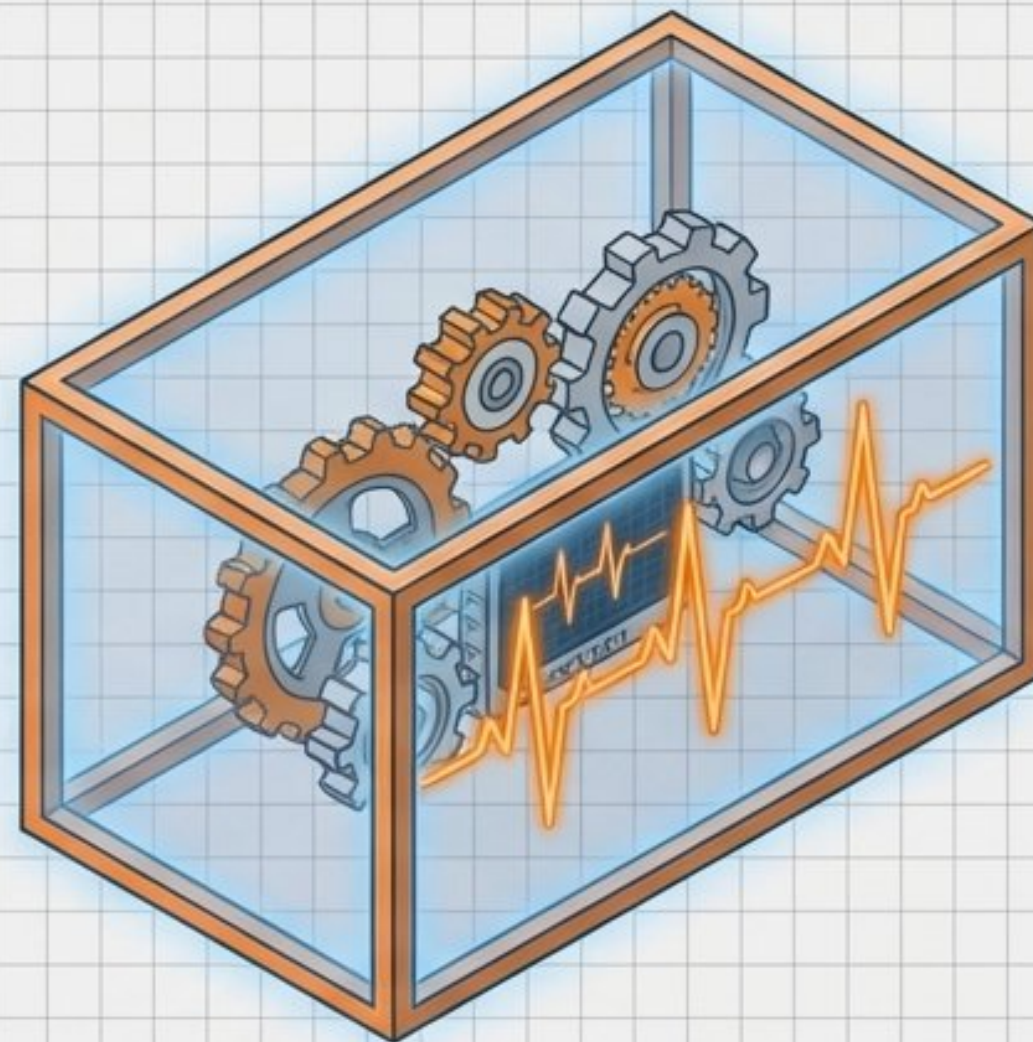
Docker Registries are stateless, scalable storage systems supporting both public open-source tools and private corporate repositories.

Phase 3: The Live Instance (Docker Container)

[Level: Stage 3 - Run]



Inert Snapshot

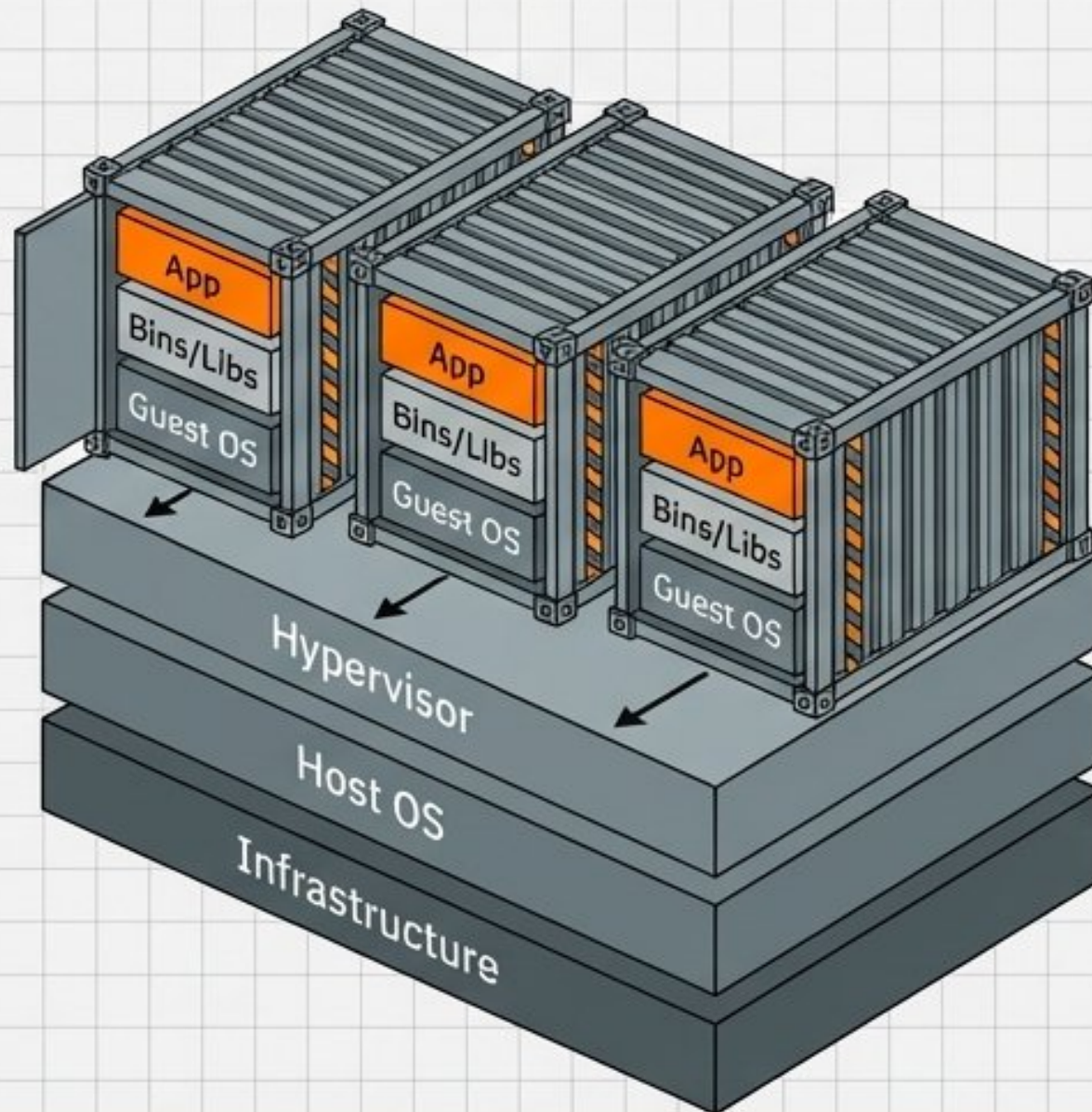


Living Application

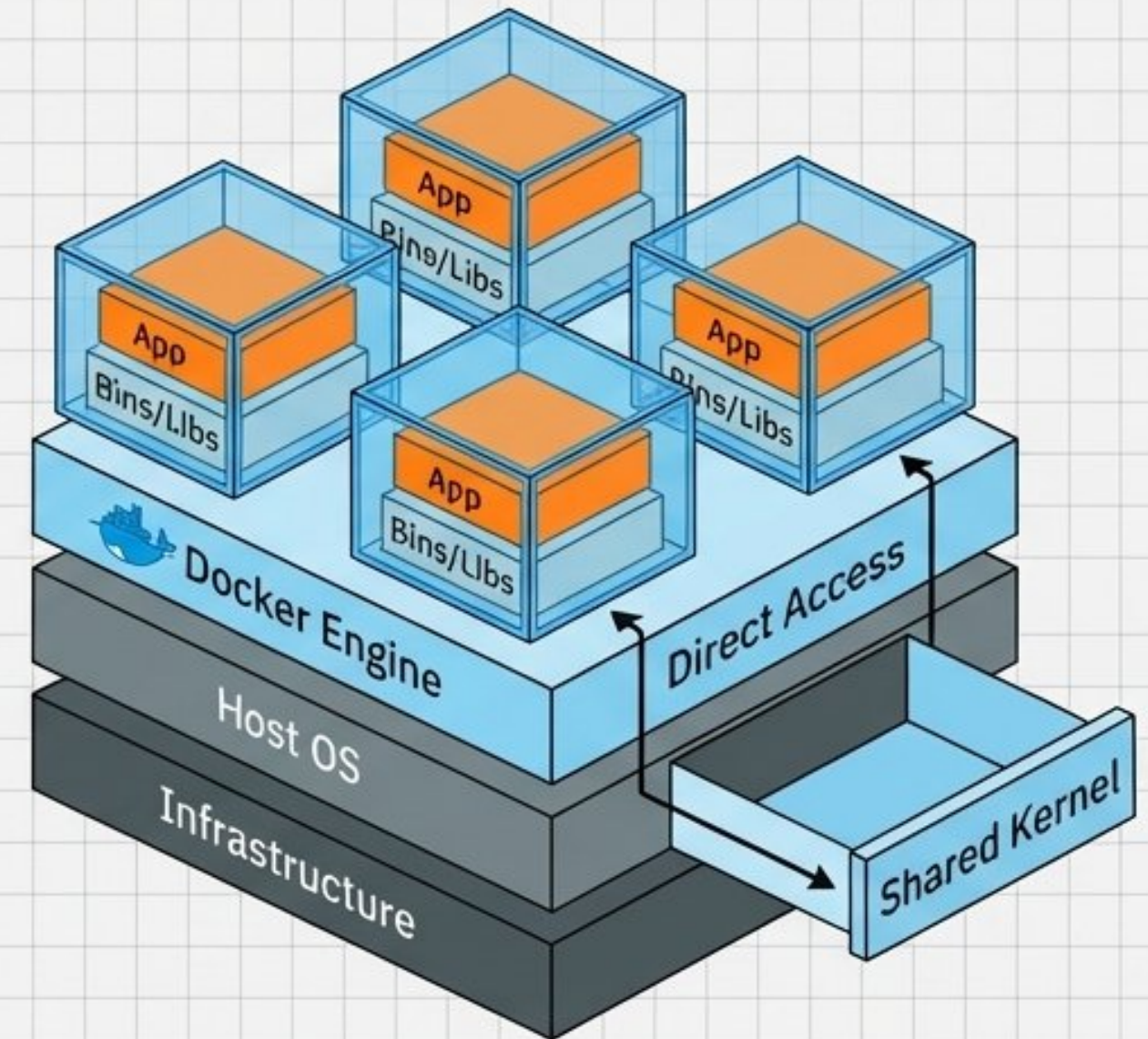
A container is a runnable, live instance of an image. If the Image is the blueprint, the Container is the physical house built from that blueprint.

- Isolated filesystem
- Dedicated networking
- Active process space

Bulky Virtual Machine



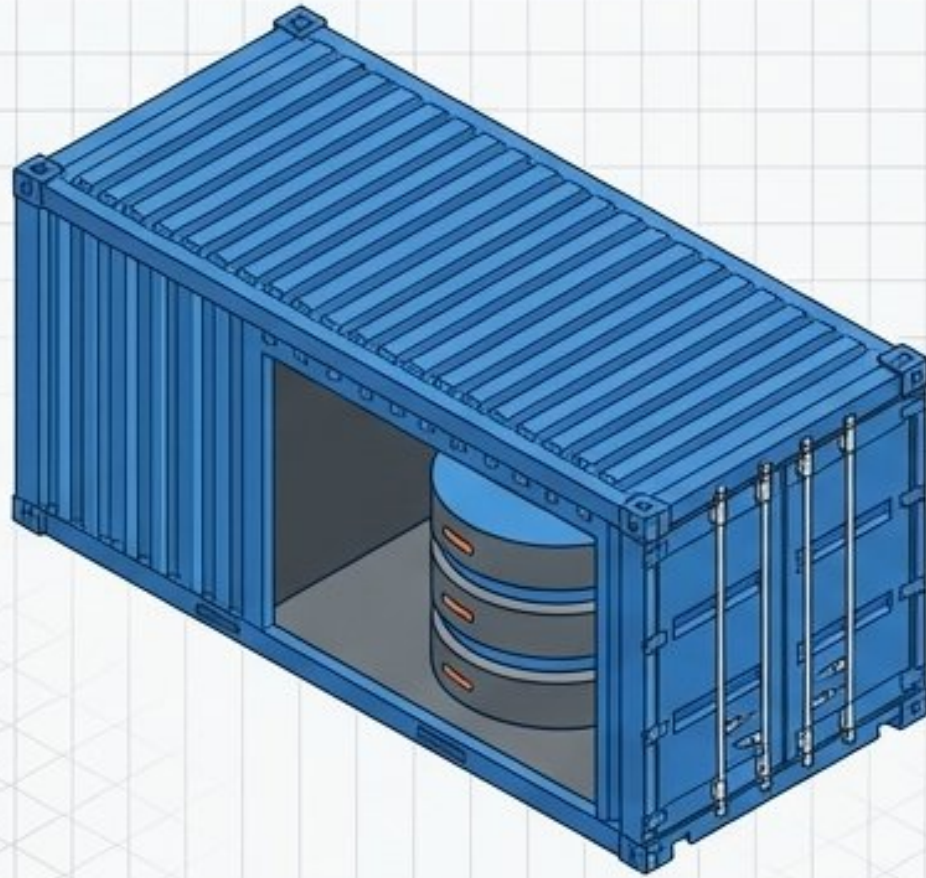
Sleek Docker Container



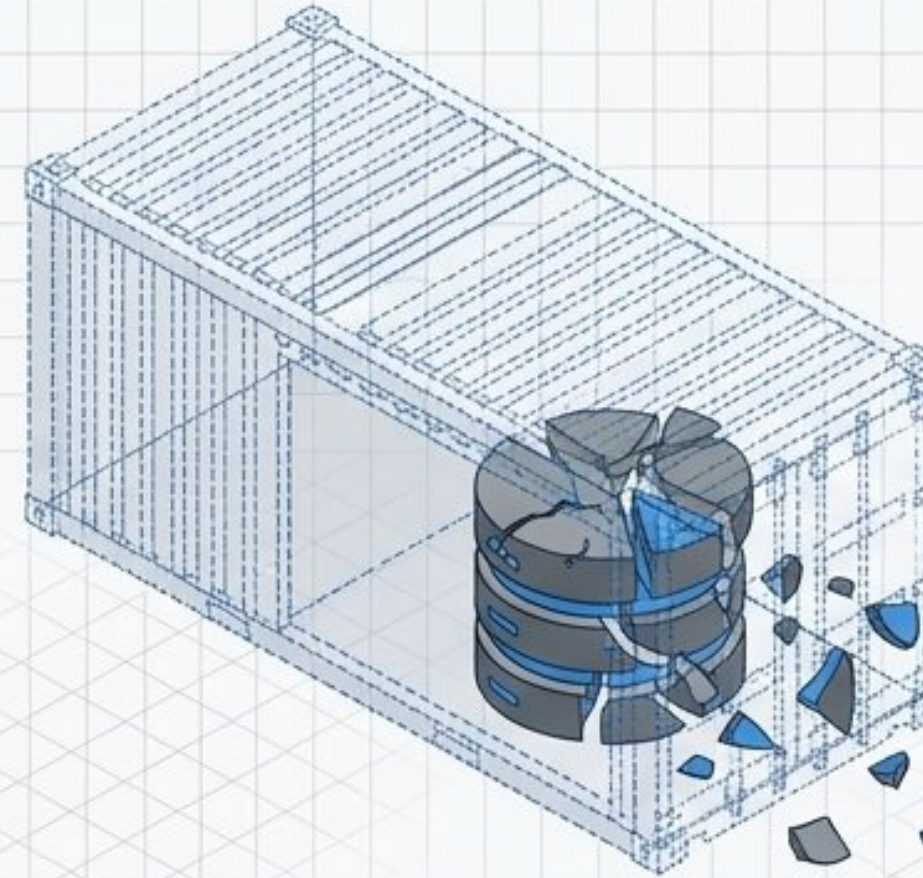
Containers share the host operating system's kernel but keep applications isolated. Result: faster startup, microscopic resource footprint, and infinite portability.

The Ephemeral Storage Trap

[Level: Storage Diagnostics]



Active Container



Deleted Container



By default, everything created inside a container is temporary. A container's writable layer is ephemeral.

If a container crashes, stops, or is deleted, the internal filesystem is wiped clean. Data is lost forever. To run stateful applications, data **must be anchored safely outside**.

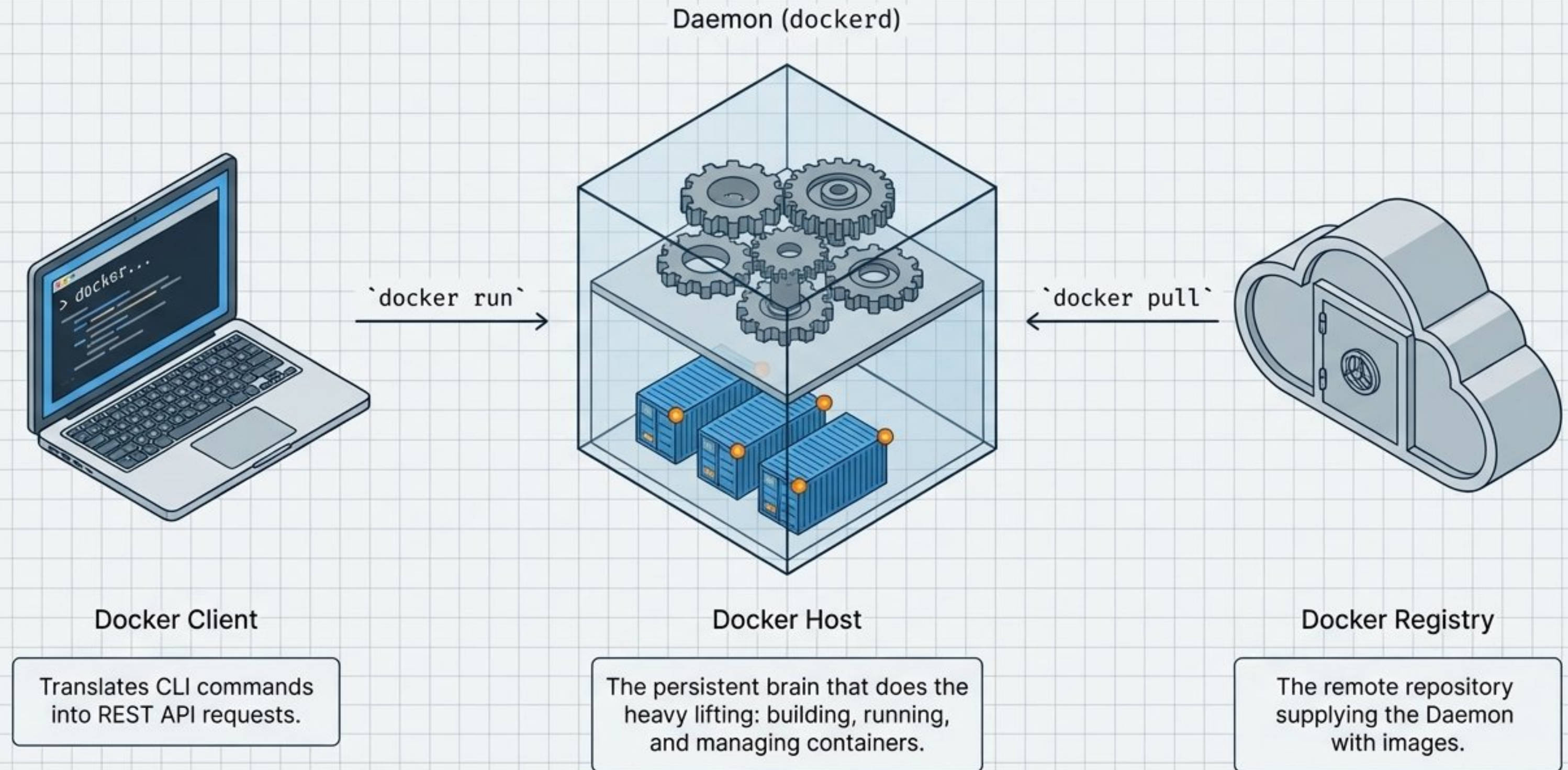
State-Transition Diagnostic Matrix

[Level: Synthesis]

	Dockerfile	Docker Image	Docker Container
Metaphor	The Blueprint	The Snapshot	The Live Instance
Format	Text / Source Code	Binary / Packaged	Active OS Process
State	Declarative	Immutable (Read-Only)	Ephemeral (Read/Write)
Primary Action	<code>`docker build`</code>	<code>`docker push / pull`</code>	<code>`docker run / start`</code>

The Docker Ecosystem Topology

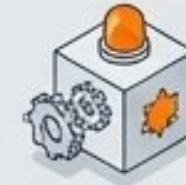
[Level: Docker Engine]



The Engineer's Console

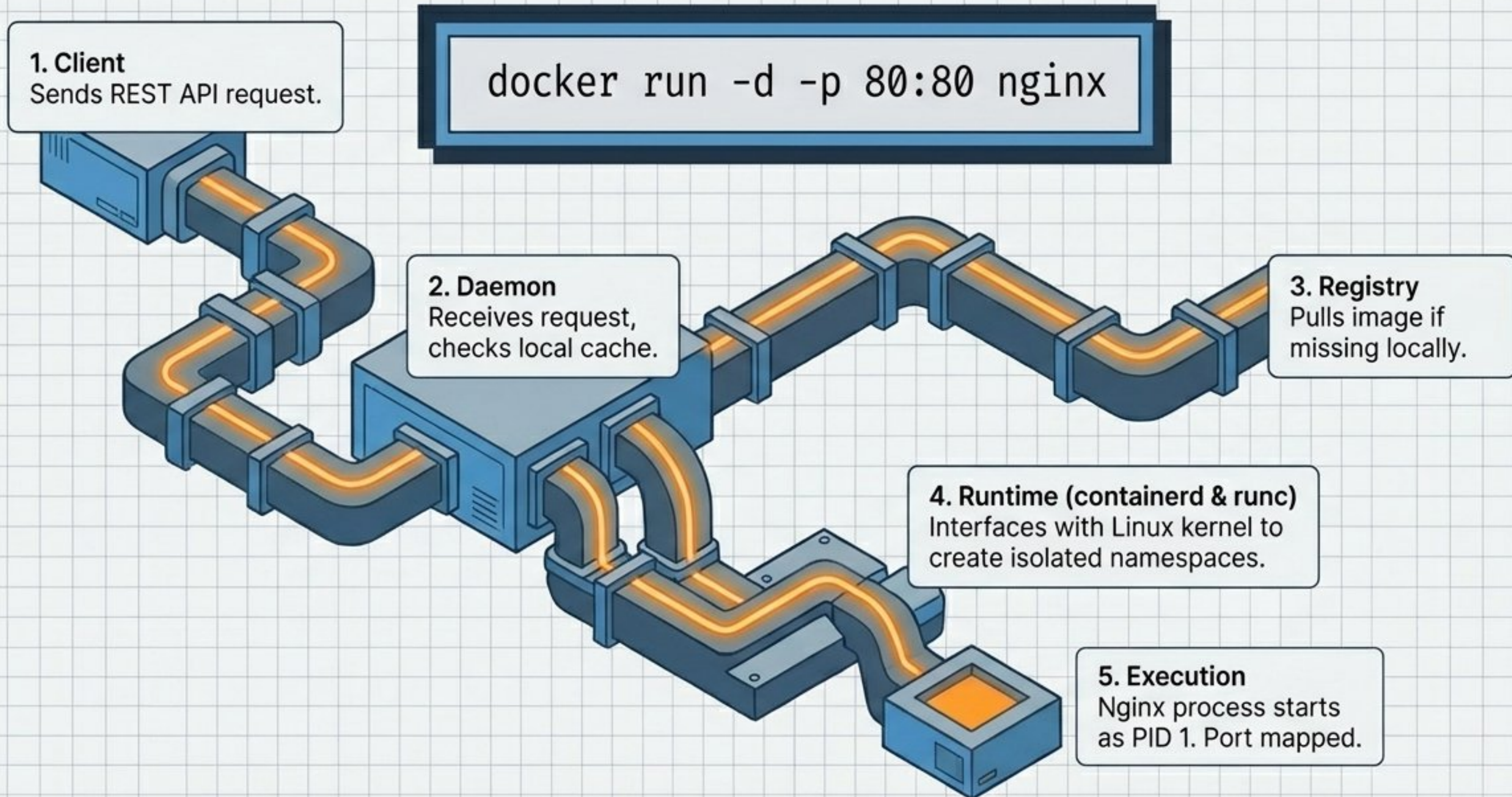
[Level: CLI Execution]

```
> docker build -t my-app .  
// Reads the Dockerfile and creates a layered image  
  
> docker image ls  
// Lists the stored frozen snapshots ready for deployment  
  
> docker run -d -p 8080:80 my-app  
// Boots the image into an isolated, running container  
  
> docker ps  
// Displays the heartbeat and status of active containers
```



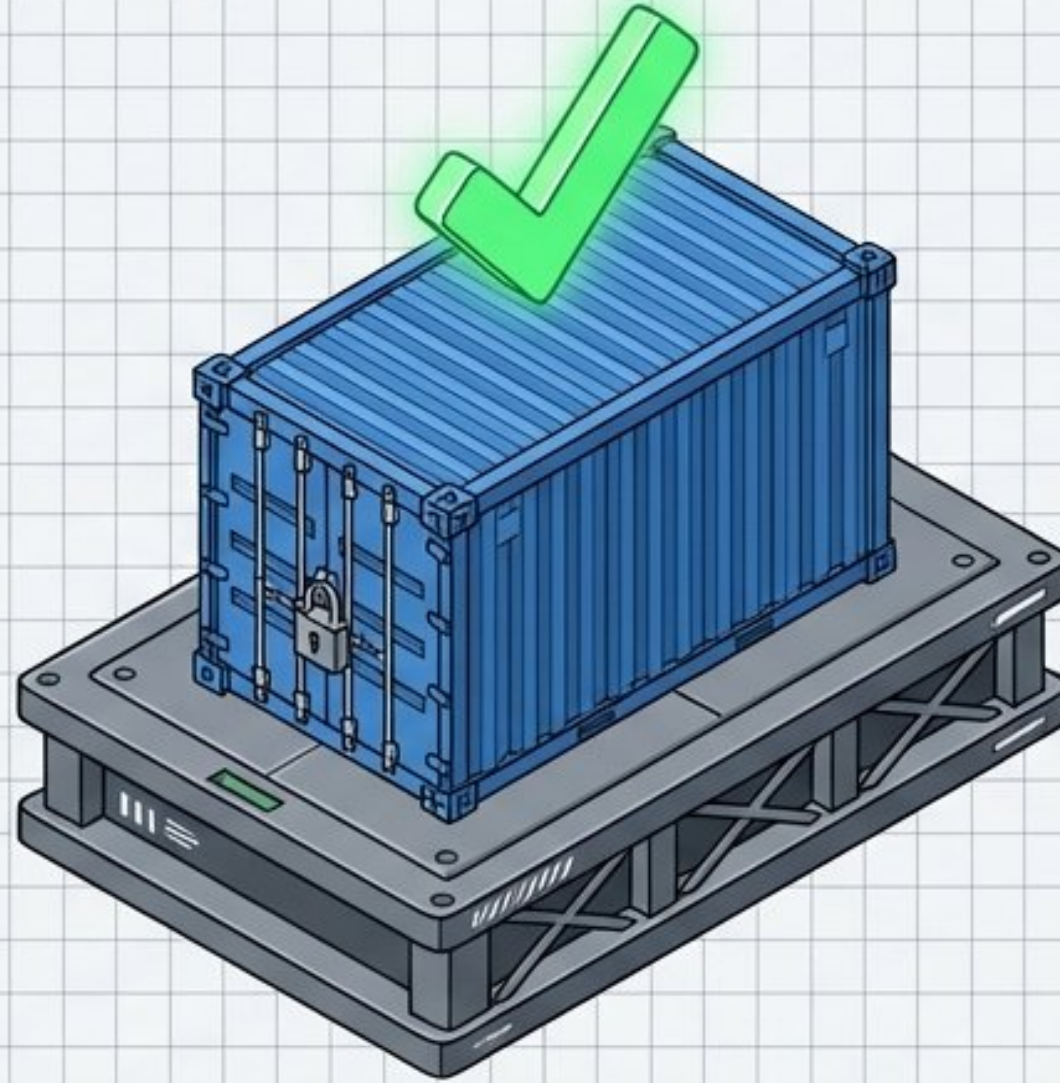
Under the Hood: Tracing `docker run`

[Level: Deep Dive]



The Immutable Golden Path

[Level: Takeaway]



Automate

Treat infrastructure as declarative code (Dockerfile).

Standardize

Distribute immutable, identical snapshots (Images).

Isolate

Run applications in secure, lightweight environments (Containers).

The 'works on my machine' era is over. Welcome to the shipyard.